# PCCCS495 : Advanced Programming(OOP) Lab Project Report

**TITLE-Modular Quiz Engine Using Factory Pattern**


**NAME-**ARUSHI DAS

**STREAM-**CSE(IOT CS BT)

**SEC-**A

**ENROLLMENT NUMBER-**12024052017049

**ROLL NUMBER-**50

**SEMESTER-** 4


**INSTRUCTOR'S NAME:**

*Dr. Swagatam Basu, Dr. Susovan Jana,Dr. Soumadip Biswas*

**SPRING 2026**

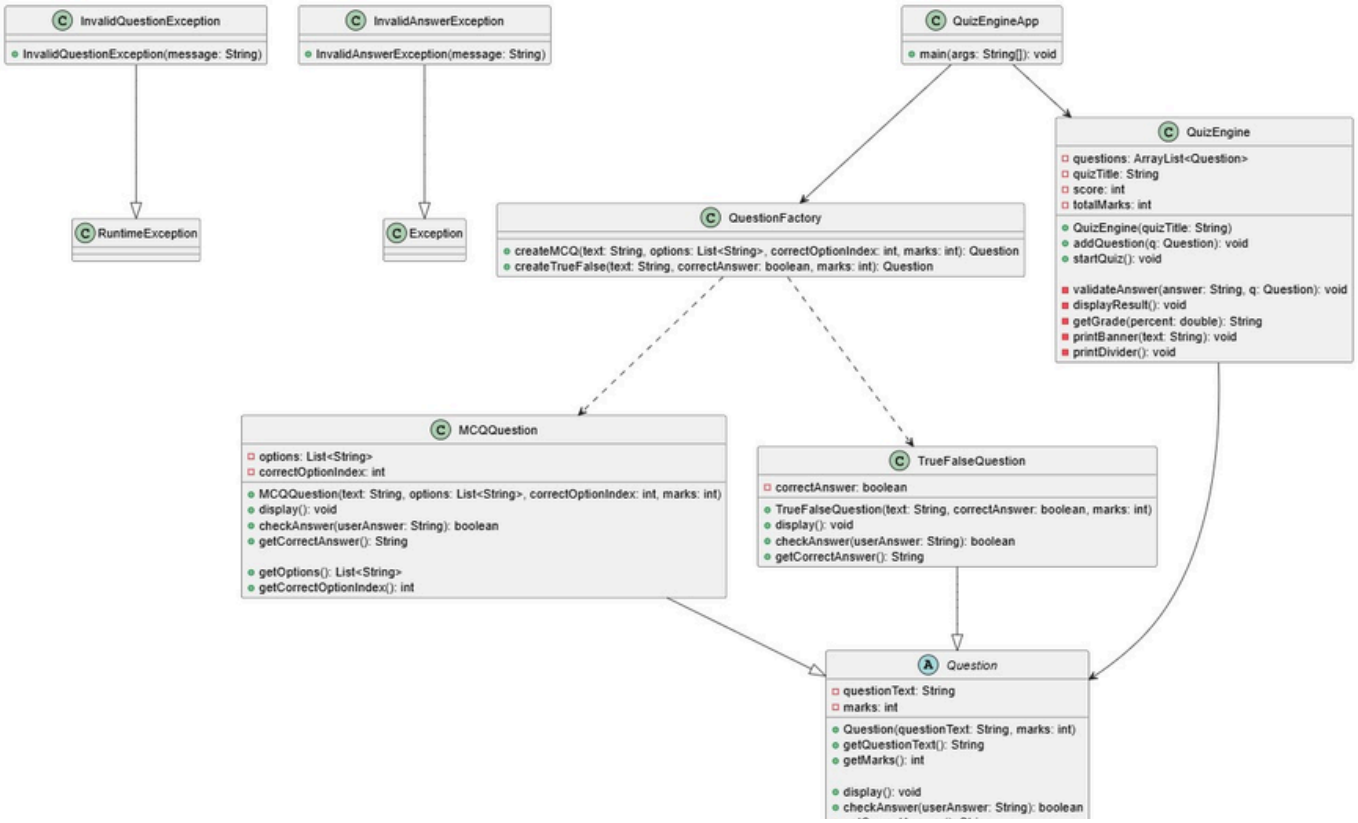
# PROBLEM STATEMENT

Many small educational institutions and student groups require a simple yet structured system to conduct quizzes with different types of questions. Most basic quiz programs mix all logic into a single class, making them difficult to extend, maintain, or modify. Adding new question types or changing evaluation rules often requires rewriting major parts of the system.

The proposed Modular Quiz Engine solves this problem by using object-oriented design principles to create a flexible and extensible quiz system. Each question type is modeled as a separate class, allowing dynamic behavior through polymorphism. A Factory Pattern is used to manage question creation, reducing tight coupling. The system ensures clean architecture, modular structure, and robust error handling while remaining simple and efficient.

# SYSTEM DESIGN

## 1)UML DIAGRAM




## 2)Description of Major Classes:

The project is based on object-oriented design with several key classes. The abstract class Question serves as the base, defining common properties like questionText and marks, along with methods for displaying and checking answers. MCQQuestion and TrueFalseQuestion extend this class to handle multiple-choice and true/false questions respectively. The QuestionFactory class uses the Factory Design Pattern to create question objects. The QuizEngine class manages the quiz flow, including adding questions, validating answers, and calculating results.

The QuizEngineApp class contains the main method and starts the application. Custom exceptions like InvalidQuestionException and InvalidAnswerException are used for error handling.

# 3)Package Structure

The project can be logically divided into packages for better organization. The exceptions package handles error classes like InvalidQuestionException and InvalidAnswerException. The model package contains core classes such as Question, MCQQuestion, and TrueFalseQuestion. The factory package includes QuestionFactory for object creation. The engine package contains QuizEngine, which manages the quiz logic, and the main package includes QuizEngineApp, the entry point of the program.

# 4)Interaction Flow

The program starts from the main method in QuizEngineApp, where a QuizEngine object is created. Questions are added using QuestionFactory, and the quiz begins with startQuiz(). Each question is displayed, user input is taken and validated, and answers are checked to update the score. After all questions are completed, the system calculates and displays the final score, percentage, and grade.

# OOP Concepts Demonstrated

## Abstraction

**Where Used:**

The abstraction concept is implemented in the Question.java file, which serves as the abstract base class for all types of questions in the application.

**Why Used:**

Abstraction is used to hide internal implementation details and expose only the necessary functionality. The QuizEngine does not need to know whether a question is an MCQ or a True/False type; it only interacts with the common methods defined in the Question class, such as displaying the question and checking the answer. This makes the code simpler, more flexible, and easier to maintain.

**Code Snippet — Question.java (abstract class):**

```java
package quizengine;

public abstract class Question {
    private String questionText;  // encapsulated field
    private int marks;

    public Question(String questionText, int marks) {
        this.questionText = questionText.trim();
        this.marks = marks;
    }


    public abstract void display();
    public abstract boolean checkAnswer(String userAnswer);
    public abstract String getCorrectAnswer();


    public String getQuestionText() { return questionText; }
    public int getMarks()           { return marks; }
}
```

# Inheritance

**Where Used:**
Inheritance is used where the MCQQuestion and TrueFalseQuestion classes extend the Question class, allowing them to reuse the common properties and methods defined in the base class.

**Why Used:**
Inheritance helps avoid code duplication by allowing both MCQQuestion and TrueFalseQuestion to share common fields like questionText and marks, as well as common methods from the Question class. Each subclass only adds features specific to its type, making the code more organized, reusable, and easier to maintain.

## Code Snippet — MCQQuestion.java:

```java
public class MCQQuestion extends Question {
    private List<String> options;
    private int correctOptionIndex;

    public MCQQuestion(String questionText, List<String> options,
                       int correctOptionIndex, int marks) {
        super(questionText, marks);  // calls Question constructor
        this.options = options;
        this.correctOptionIndex = correctOptionIndex;
    }

    @Override
    public boolean checkAnswer(String userAnswer) {
        char ch = userAnswer.trim().toUpperCase().charAt(0);
        return (ch - 'A') == correctOptionIndex;
    }
}
```

# Polymorphism

**Where Used:**
Polymorphism is used in the QuizEngine class, specifically in the startQuiz() method, where methods like display() and checkAnswer() are called using a Question reference.

**Why Used:**
Polymorphism allows a single piece of code to work with different types of objects. The QuizEngine stores all questions as Question objects and calls the same methods on each one. At runtime, Java automatically decides whether to execute the MCQQuestion or TrueFalseQuestion implementation, making the code flexible and easy to extend.

Code Snippet — QuizEngine.java (startQuiz method):

```
for (int i = 0; i < questions.size(); i++) {
    Question q = questions.get(i);  // Question reference — type unknown

    q.display();          // MCQQuestion: prints A/B/C/D options
                          // TrueFalseQuestion: prints True / False

    String userAnswer = scanner.nextLine().trim();

    if (q.checkAnswer(userAnswer)) {  // each subclass checks differently
        score += q.getMarks();
        System.out.println("  Correct! +" + q.getMarks());
    } else {
        System.out.println("  Wrong. Correct: " + q.getCorrectAnswer());
    }
}
```

# Encapsulation

**Where Used:**

Encapsulation is used throughout the project as all fields in classes like Question, MCQQuestion, TrueFalseQuestion, and QuizEngine are declared as private.

**Why Used:**

Encapsulation protects the internal state of objects from direct external access. Data can only be accessed through getters or controlled methods, which helps prevent accidental modification and ensures better data security and control.

Code Snippet — TrueFalseQuestion.java:

```java
public class TrueFalseQuestion extends Question {

    private boolean correctAnswer;

    public TrueFalseQuestion(String questionText,
                             boolean correctAnswer, int marks) {
        super(questionText, marks);
        this.correctAnswer = correctAnswer;
    }


    public boolean isCorrectAnswer() { return correctAnswer; }

    @Override
    public String getCorrectAnswer() {
        return correctAnswer ? "True" : "False";
    }
}
```

# Exception Handling

**Where Used**:
Exception handling is implemented using InvalidAnswerException in the validateAnswer() method of QuizEngine and InvalidQuestionException in the constructor of the Question class.

**Why Used:**
Exception handling separates error-handling logic from normal program flow. Instead of crashing on invalid input, the program catches exceptions, displays a proper message, and allows the user to try again, improving reliability.

Code Snippet — QuizEngine.java (input validation loop):

```
while (!validInput) {
    try {
        userAnswer = scanner.nextLine().trim();
        validateAnswer(userAnswer, q);
        validInput = true;
    } catch (InvalidAnswerException e) {

        System.out.print("  [!] " + e.getMessage() + " Try again: ");
    }
}


private void validateAnswer(String answer, Question q)
        throws InvalidAnswerException {
    if (answer == null || answer.isEmpty()) {
        throw new InvalidAnswerException("Answer cannot be empty.");
    }
}
```

# Collections (ArrayList)

**Where Used:**

The QuizEngine class uses ArrayList<Question> to store all quiz questions.

**Why Used:**

ArrayList allows dynamic storage of questions, maintains order, and provides easy access during iteration. It removes the need to define a fixed size in advance, making the program more flexible.

Code Snippet — QuizEngine.java:

```
private ArrayList<Question> questions;  // stores mixed types polymorphically

public void addQuestion(Question q) {
    if (q == null) {
        throw new InvalidQuestionException("Cannot add a null question.");
    }
    questions.add(q);            // dynamic add — no size limit
    totalMarks += q.getMarks(); // accumulate total as we add
}

// During quiz — iterate with index for question numbering
for (int i = 0; i < questions.size(); i++) {
```

```
    Question q = questions.get(i);
    // ... display and check
}
```

# Factory Design Pattern

**Where Used:**

The Factory Design Pattern is used in the QuestionFactory class through methods like createMCQ() and createTrueFalse(), which are called from the main class.

**Why Used:**

The factory pattern separates object creation from business logic. The main class does not directly create objects using constructors, making the code more flexible and easier to maintain if object creation logic changes in the future.

Code Snippet — QuestionFactory.java:

```java
public class QuestionFactory {

    // Static factory method — centralised creation
    public static Question createMCQ(String text,
            List<String> options, int correctIdx, int marks) {
        return new MCQQuestion(text, options, correctIdx, marks);
    }

    public static Question createTrueFalse(
            String text, boolean correct, int marks) {
        return new TrueFalseQuestion(text, correct, marks);
    }
}

// Main.java — clean usage with no 'new MCQQuestion()' directly
engine.addQuestion(QuestionFactory.createMCQ(
    "Which keyword creates a subclass?",
    Arrays.asList("implements","extends","inherits","super"), 1, 2));
```

# Implementation Highlights

## Highlight 1 — Abstract Contract Enforcement

| | |
|---|---|
| **Justification** | Declaring Question as abstract with three abstract methods guarantees that every new question type added in future will be forced by the compiler to implement display(), checkAnswer(), and getCorrectAnswer(). Without this, a developer could accidentally add a question type with missing behaviour and only discover the bug at runtime. |

```java

public abstract class Question {
    public abstract void display();
    public abstract boolean checkAnswer(String userAnswer);
    public abstract String getCorrectAnswer();


}
```

## Highlight 2 — Input Validation with Re-Prompt Loop

| | |
|---|---|
| **Justification** | A quiz that crashes on bad input is unusable. The while loop with try-catch keeps asking until valid input is received. The exception carries a descriptive message so the user knows exactly what went wrong. This pattern is the standard approach for safe console input in Java. |

```java

private void validateAnswer(String answer, Question q)
        throws InvalidAnswerException {
    if (answer == null || answer.isEmpty())
        throw new InvalidAnswerException("Answer cannot be empty.");
    if (q instanceof MCQQuestion) {
        char c = answer.toUpperCase().charAt(0);
        int n  = ((MCQQuestion) q).getOptions().size();
        char max = (char)('A' + n - 1);
        if (c < 'A' || c > max)
            throw new InvalidAnswerException(
                "Enter a letter between A and " + max + ".");
    }
}
```

## Highlight 3 — Constructor-Level Validation

| | |
|---|---|
| **Justification** | Questions with empty text or zero marks are meaningless. By validating inside the Question constructor and throwing InvalidQuestionException immediately, we guarantee that if a Question object exists, it is always in a valid state. This is the 'fail fast' principle — errors are caught at the earliest possible point. |

```java

public Question(String questionText, int marks) {
    if (questionText == null || questionText.trim().isEmpty())
        throw new InvalidQuestionException(
            "Question text cannot be empty.");
    if (marks <= 0)
        throw new InvalidQuestionException(
            "Marks must be a positive integer.");
    this.questionText = questionText.trim();
    this.marks = marks;

}
```

## Highlight 4 — Grade Calculation with Guard Clauses

| | |
|---|---|
| **Justification** | The getGrade() method uses sequential guard clauses (if/return) rather than if/else-if chains. This is cleaner and easier to read. Each condition is checked from highest to lowest and returns immediately on a match, making it trivial to add or adjust grade boundaries. |

```java

private String getGrade(double percent) {
    if (percent >= 90) return "A+ (Outstanding)";
    if (percent >= 75) return "A  (Excellent)";
    if (percent >= 60) return "B  (Good)";
    if (percent >= 45) return "C  (Average)";
    if (percent >= 33) return "D  (Pass)";
    return "F  (Fail)";
}


double percent = (totalMarks > 0) ? (score * 100.0 / totalMarks) : 0;
System.out.printf("  Percent : %.1f%%\n", percent);
System.out.println("  Grade   : " + getGrade(percent));
```

## Highlight 5 — Case-Insensitive Multi-Format Answer Acceptance

| | |
|---|---|
| **Justification** | Users should not fail a question just because they typed 'true' instead of 'True', or 'b' instead of 'False'. TrueFalseQuestion.checkAnswer() accepts six equivalent forms of each answer. This improves user experience without changing the question logic. |

```java

@Override
public boolean checkAnswer(String userAnswer) {
    if (userAnswer == null || userAnswer.trim().isEmpty()) return false;
    String a = userAnswer.trim().toLowerCase();
    boolean userBool;
    switch (a) {
        case "true":  case "t": case "a": userBool = true;  break;
        case "false": case "f": case "b": userBool = false; break;
        default: return false;  // unrecognised input = wrong
    }
    return userBool == correctAnswer;
```

# Testing & Error Handling

## Functional Test Cases

| Test Case | Input | Expected Output | Result |
|---|---|---|---|
| MCQ — correct answer | `Input: B (correct index = 1)` | Score increases by marks | PASS |
| MCQ — wrong answer | `Input: A (correct index = 1)` | Score unchanged, answer shown | PASS |
| MCQ — lowercase input | `Input: b` | Treated same as B — correct | PASS |
| T/F — correct (True) | `Input: True` | Score increases | PASS |
| T/F — input as A/B | `Input: A (for True)` | Accepted, correct | PASS |
| T/F — input as t/f | `Input: f (for False)` | Accepted, correct | PASS |
| Score calculation | `7 correct out of 14 marks` | Score = 7, 50.0%, Grade B | PASS |
| Grade — 100% score | `All 14 marks correct` | Grade = A+ Outstanding | PASS |
| Grade — 0% score | `All questions wrong` | Grade = F Fail | PASS |
| 10 questions displayed | `startQuiz() called` | All 10 questions appear in order | PASS |

## Edge Cases Handled

| Test Case | Input | Expected Output | Result |
|---|---|---|---|
| Empty answer input | `Input: (just Enter key)` | InvalidAnswerException thrown, re-prompt | PASS |
| Whitespace-only input | `Input: '  '` | Trimmed to empty, exception thrown | PASS |
| MCQ — out-of-range letter | `Input: E (only A-D valid)` | InvalidAnswerException, re-prompt | PASS |
| MCQ — number input | `Input: 2` | Invalid, exception thrown | PASS |
| T/F — random word | `Input: maybe` | Returns false (unrecognised) | PASS |
| T/F — mixed case | `Input: TRUE or TrUe` | Lowercased before check — accepted | PASS |
| Zero totalMarks guard | `totalMarks = 0 (no questions)` | Percent = 0.0, no division error | PASS |

## Failure Scenarios & How They Are Handled

| Failure Scenario | Exception / Mechanism | Outcome |
|---|---|---|
| Question created with empty text | `InvalidQuestionException (runtime)` | Program refuses to start — caught at object creation time |
| Question created with marks <= 0 | `InvalidQuestionException (runtime)` | Same as above — invalid state never reaches the quiz loop |
| MCQ correct index out of range | `InvalidQuestionException (runtime)` | Constructor rejects the object before it is stored |
| User enters blank answer | `InvalidAnswerException (checked)` | Caught in while loop — user is asked to try again, quiz continues |
| User enters invalid MCQ letter | `InvalidAnswerException (checked)` | Caught in while loop — descriptive message shown, re-prompted |
| User enters unrecognised T/F input | `Returns false from checkAnswer()` | Treated as wrong answer — correct answer is displayed |
| No questions added to engine | `Guard in startQuiz()` | Prints error message and returns without starting quiz |
| null question passed to addQuestion() | `InvalidQuestionException (runtime)` | Exception thrown immediately before null is stored in the list |

# Git Workflow

## Screenshot of commit history

| Name | Last commit message | Last commit date |
|---|---|---|
| .. | | |
| .keep | Initial commit | 27 days ago |
| InvalidAnswerException.java | Results for invalid answers | 16 hours ago |
| InvalidQuestionException.java | for invalid questions | 15 hours ago |
| MCQQuestion.java | add MCQQuestion class with validation and answer checking | 2 hours ago |
| Main.java | Implement QuizEngine with multiple MCQ and True/False questions for J... | 2 hours ago |
| Question.java | add abstract Question base class with validation and core methods | 28 minutes ago |
| QuestionFactory.java | question factory set | 2 days ago |
| QuizEngine.java | quiz engine completed | 3 days ago |
| TrueFalseQuestion.java | add TrueFalseQuestion class with answer validation | 1 hour ago |

## Development Progression:

The project was developed in a structured and incremental manner. It began with the initial setup, followed by the creation of custom exception classes to handle invalid inputs. Next, the abstract Question class was implemented to define the core structure and common behavior for all question types. After that, specific question types like MCQQuestion and TrueFalseQuestion were developed with proper validation and answer-checking logic. The QuestionFactory class was then introduced to simplify object creation using the Factory Design Pattern. Once the question models were ready, the QuizEngine class was built to manage the quiz flow, including displaying questions, validating answers, and calculating scores. Finally, the Main class was implemented to integrate all components, add sample questions, and run the complete quiz system. This step-by-step approach ensured modular development, easier debugging, and better code organization.

# Conclusion:

The project successfully demonstrates the use of object-oriented programming concepts such as abstraction, inheritance, polymorphism, encapsulation, and design patterns like the Factory Pattern. It provides a modular and scalable quiz system that can handle different types of questions, validate user input, and calculate results efficiently. The use of proper exception handling ensures robustness and improves user experience. Overall, the project achieves its goal of creating a flexible and well-structured quiz application.

# Future Scope:

The project can be further enhanced by adding a graphical user interface (GUI) to improve usability, integrating a database to store questions and user scores, and supporting more question types such as fill-in-the-blanks or descriptive answers. Additional features like timer-based quizzes, user authentication, and performance analytics can also be implemented to make the system more advanced and practical for real-world applications.